Assignment 7 – Soln

```
class ALSU_test extends uvm_test;
  `uvm_component_utils(ALSU_test)

  ALSU_env env;
  ALSU_config ALSU_cfg;
  ALSU_main_sequence main_seq;
  ALSU_reset_sequence reset_seq;
  shift_reg_env env_shift;
  shift_reg_config shift_reg_cfg;

  function new(string name = "ALSU_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = ALSU_env::type_id::create("env",this);
    env_shift = shift_reg_env::type_id::create("env_shift", this);
    ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg",this);
    shift_reg_cfg = shift_reg_config::type_id::create("shift_reg_cfg",this);
    main_seq = ALSU_main_sequence::type_id::create("main_seq",this);
    reset_seq = ALSU_reset_sequence::type_id::create("reset_seq",this);
    set_type_override_by_type(ALSU_seq_item::get_type(), ALSU_seq_item_valid_invalid::get_type());

    if (!uvm_config_db #(virtual ALSU_if)::get(this, "","ALSU_IF", ALSU_cfg.ALSU_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the ALSU from the uvm_config_db");
    if (!uvm_config_db #(virtual shift_reg_if)::get(this, "","shift_reg_IF", shift_reg_cfg.shift_reg_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the shift_reg from the uvm_config_db");

    shift_reg_cfg.is_active = UVM_PASSIVE;
    ALSU_cfg.is_active = UVM_ACTIVE;

    uvm_config_db #(shift_reg_config)::set(this, "*", "CFG_SHIFT", shift_reg_cfg);
    uvm_config_db #(ALSU_config)::set(this, "*", "CFG_ALSU", ALSU_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    //reset sequence
    `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)

    //main sequence
    `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
    main_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
    phase.drop_objection(this);
  endtask
endclass: ALSU_test
```

```
class ALSU_seq_item extends uvm_sequence_item;
 `uvm_object_utils(ALSU_seq_item)

  rand bit clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, serial_in;
  rand direction_e direction;
  rand opcode_e opcode;
  rand opcode_valid_e opcode_valid;
  rand bit signed [2:0] A, B;
  logic [15:0] leds;
  logic [5:0] out;

  function new(string name = "ALSU_seq_item");
  super.new(name);
  endfunction

  function string convert2string();
    return $sformatf("%s reset = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b, bypass_A = 0b%b,
      bypass_B = 0b%b,  serial_in = 0b%b, direction = %s,  opcode = %s, A = 0b%b, B = 0b%b, leds = 0b%b, out = 0b%b",
      super.convert2string(), rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, serial_in, direction, opcode, A, B, leds, out);
  endfunction

  function string convert2string_stimulus();
    return $sformatf("%s reset = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b, bypass_A = 0b%b,
      bypass_B = 0b%b,  serial_in = 0b%b, direction = %s,  opcode = %s, A = 0b%b, B = 0b%b",
      super.convert2string(), rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, serial_in, direction, opcode, A, B);
  endfunction

  constraint rst_con {
  rst dist {1:= 2, 0:= 98};
  }

  constraint opcode_con {
    opcode == opcode_valid;
  }

  constraint reduction_con {
    if (opcode != OR || opcode != XOR) {
      red_op_A == 0;
      red_op_B == 0;
    }
  }

endclass
```

```
class ALSU_seq_item_valid_invalid extends ALSU_seq_item;
 `uvm_object_utils(ALSU_seq_item_valid_invalid)

  function new(string name = "ALSU_seq_item_valid_invalid");
  super.new(name);
  endfunction

  constraint opcode_con {
    opcode dist {[OR:ROTATE]:= 80, [INVALID_6:INVALID_7]:= 20};
  }

  constraint reduction_con {
    red_op_A dist {1:= 20, 0:= 80};
    red_op_B dist {1:= 20, 0:= 80};
  }
endclass
```